



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

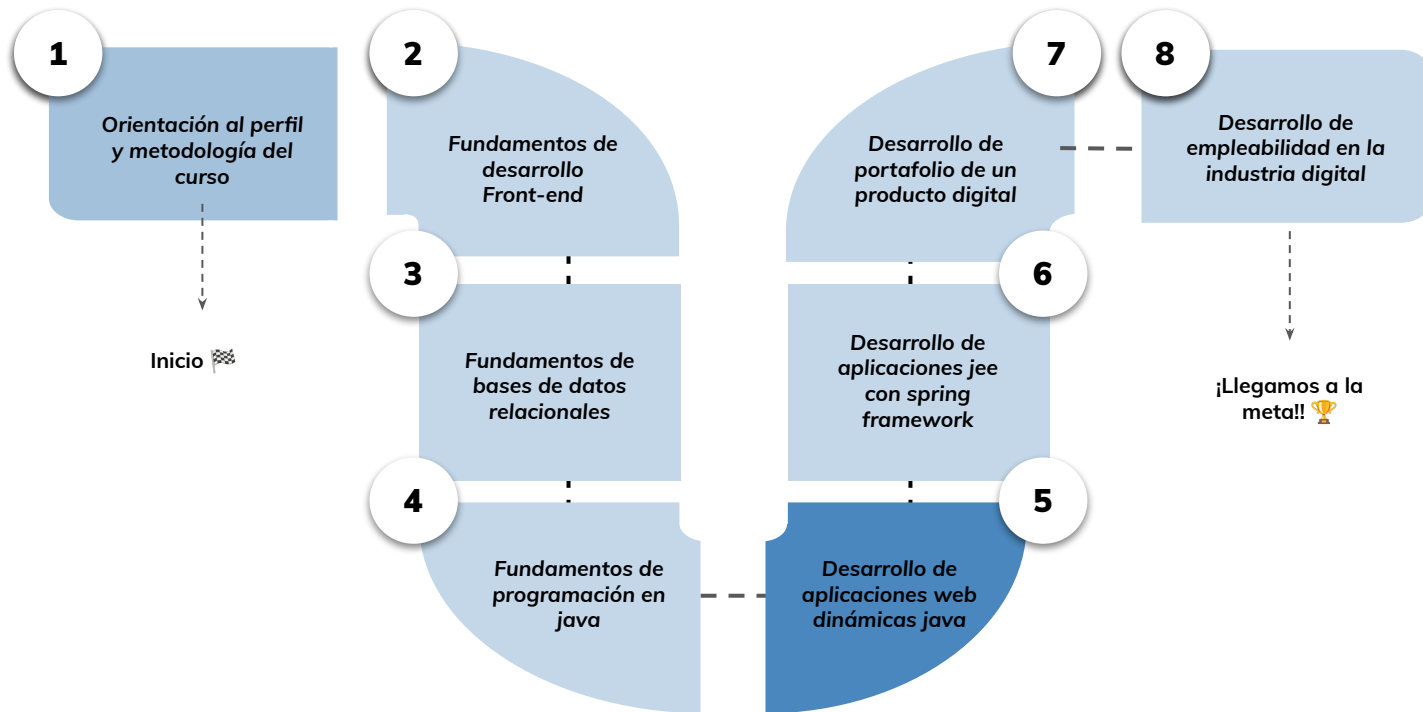


› Conociendo los servlets- Parte 1

Aprendizaje Esperado 3: Estructurar el comportamiento de una aplicación web dinámica utilizando servlets que procesan peticiones GET y POST para dar solución a un problema.

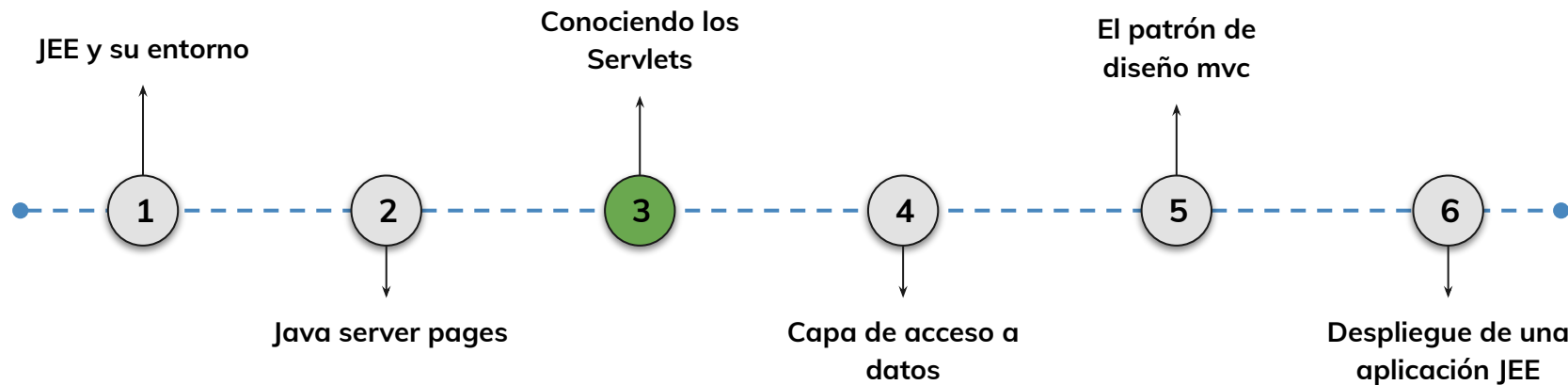
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Conociendo los Servlets

Revisaremos la introducción a los Servlets y Contenedores Web la concurrencia con los Servlets y el control de parámetros de un GET request

Servlets

GET request

Introducción a los servlets y contenedores web.

Compartiendo información entre Servlets

Controlando parámetros de un get request.

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender el concepto de Servlets y Contenedores Web
- Aprender el manejo de sesiones y cookies
- Aprender a compartir información entre servlets
- Comprender el concepto de concurrencia
- Aprender a controlar parámetros de un GET request

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprender la utilización de la etiqueta `c:forEach`
- Entender la implementación de las funciones JSTL
- Comprender la captura de datos de un formulario utilizando JSP.

➤ Servlets

< > Servlets

¿Qué es?:

Es un programa Java que **se ejecuta en un servidor Web y construye o sirve páginas web**. De esta forma se pueden construir páginas dinámicas, basadas en diferentes fuentes variables: datos proporcionados por el usuario, fuentes de información variable (páginas de noticias, por ejemplo), o programas que extraigan información de bases de datos.

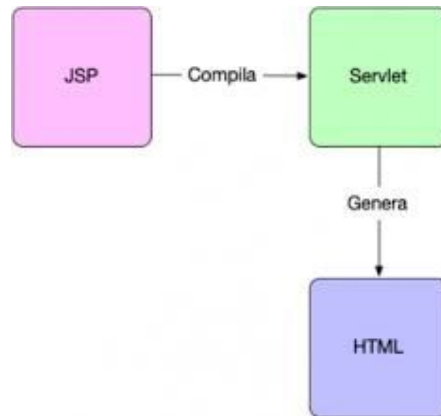
Los servlets son sencillos de utilizar, más eficientes (se arranca un hilo por cada petición y no un proceso entero), más potente y portable. Con los servlets podremos, entre otras cosas, **procesar, sincronizar y coordinar múltiples peticiones de clientes**, reenviar peticiones a otros servlets u otros servidores, etc.



< > Servlets

¿Cuál es la función de un Servlet?

1. Tomar la solicitud del cliente y devolver una respuesta.
2. La solicitud adjunta datos, y el código del Servlet sabe como recuperarlos y como usarlos.
3. La respuesta adjunta la información que el navegador necesita para mostrar la página, y el Servlet es quien envía esta información.
4. El Servlet puede decidir enviar la solicitud a otra página, Servlet o JSP.

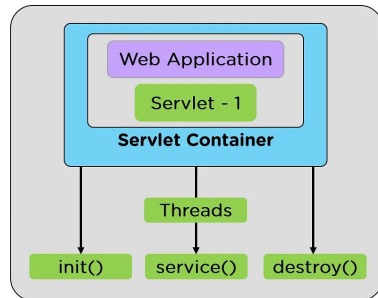


< > Servlets

Ciclo de vida de un servlet

Todos los servlets tienen el mismo ciclo de vida:

1. **Inicialización** : un servidor carga e inicializa el servlet.
2. **Procesamiento de peticiones** : el servlet procesa cero o más peticiones de clientes (por cada petición se lanza un hilo).
3. **Destrucción** : el servidor destruye el servlet (en un momento dado o cuando se apaga).



Vamos a ver estos pasos en profundidad.

< > Servlets

1- Inicialización:

Los servlet tienen un inicializador por defecto en el método `init()`.

- `public void init() throws ServletException{ ... }`
- `public void init(ServletConfig conf) throws ServletException{
 super.init(conf); ...}`

El primer método se utiliza si el servlet no necesita parámetros de configuración externos. El segundo se emplea para tomar dichos parámetros.

Si queremos definir nuestra propia inicialización, deberemos **sobreescribir** alguno de estos métodos. Podemos utilizar la inicialización para establecer una conexión con una base de datos, abrir ficheros, o cualquier tarea que se necesite hacer una sola vez antes de que el servlet comience a funcionar.



< > Servlets

2. Procesamiento de peticiones

Una vez inicializado, cada petición de usuario lanza un hilo que llama al método `service()` del servlet.

```
public void service(HttpServletRequest request, HttpServletResponse response) throws  
ServletException, IOException
```

Este método obtiene el tipo de petición que se ha realizado (GET, POST, PUT, DELETE). Dependiendo del tipo de petición que se tenga, se llama luego a uno de los métodos:

- **doGet():** Para peticiones de tipo GET (aquellas realizadas al escribir una dirección en un navegador, pinchar un enlace o rellenar un formulario que no tenga METHOD=POST)
- **doPost() :** Para peticiones POST (aquellas realizadas al rellenar un formulario que tenga METHOD=POST)



< > Servlets

3. Destrucción

El método `destroy()` de los servlets se emplea para eliminar un servlet y sus recursos asociados.

`public void destroy() throws ServletException`

Aquí debe deshacerse cualquier elemento que se construyó en la inicialización (cerrar conexiones con bases de datos, cerrar ficheros, etc). El servidor llama a `destroy()` cuando todas las llamadas de servicios del servlet han concluido, o cuando haya pasado un determinado número de segundos (lo que ocurra primero).



< > Servlets

Para convertir una clase de Java en un Servlet y que funcione como tal, basta con agregar la anotación `@WebServlet` y luego entre paréntesis la URL a la que responderá este servlet.

Además, la clase debe **extender de `HttpServlet`** .

```
13 @WebServlet("/productServlet")
14
15 public class ProductServlet extends HttpServlet {
16
```



< > Servlets

Estructura básica de un servlet:

```
@WebServlet("/productServlet")  
  
public class ProductServlet extends HttpServlet {  
  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
  
        //... codigo para peticion Get  
  
    }  
  
    protected void doPost(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
  
        //... codigo para peticion Post  
  
    }  
}
```


LIVE CODING

Ejemplo en vivo

Creando Servlets:

Vamos a crear un Servlet básico con los métodos vacíos para comenzar a practicar:

1- Crear un servlet WalletServlet que tenga declarados métodos para procesar solicitudes GET y POST.

Tiempo: 15 minutos

➤ Manejando las Sesiones

< > Manejando las sesiones

Las sesiones contienen información del usuario activo, como su nombre o contraseña.

Una buena alternativa para no manejar información sensible de nuestros usuarios del lado del cliente son las sesiones, que no es otra cosa que una estructura de datos que es accedida exclusivamente del lado del servidor. Si sus sistemas son muy concurridos o están detrás de un balanceador de carga, manejar sesiones podría representar un reto interesante de resolver, sin embargo, la mayoría de las veces funciona muy bien con su configuración por defecto.

Veamos un ejemplo práctico de cómo manipular la sesión en nuestro servlet.



< > Manejando las sesiones

Método ejemplo:

```
private static final void home (HttpServletRequest request, HttpServletResponse response) {  
    HttpSession session = request.getSession();  
    session.setAttribute("usuario", "estudiante");  
}
```

Hemos creado una sesión y le hemos insertado un registro que describe un nombre de usuario. Luego podemos acceder a esta sesión en un request completamente diferente.

```
private static final void panel (HttpServletRequest request, HttpServletResponse response) {  
    HttpSession session = request.getSession();  
    Object usuario = (String) session.getAttribute("usuario");  
}
```

Aquí obtenemos el nombre de usuario. Vemos que, como el **getAttribute** devuelve un objeto, lo casteamos al tipo correcto para su posterior utilización.



< > Manejando las sesiones

Finalmente si quisiéramos borrar (invalidar) la sesión, podemos hacerlo de la siguiente manera:

```
private static final void salir(HttpServletRequest request, HttpServletResponse response) {  
    HttpSession sesion = request.getSession();  
    sesion.invalidate();  
}
```

El método `invalidate()` destruye la sesión y sus contenidos.



➤ Manejando las Cookies

< > Manejando las cookies

Crear una cookie es bastante fácil cuando utilizamos la **clase Cookie** que encontramos en el paquete `javax.servlet.http`, simplemente hacemos una instancia de esta clase y la mandamos en la respuesta del servidor con el método `addCookie()`.

```
private static final void galleta (HttpServletRequest request, HttpServletResponse response)
{

    Cookie galletaColor = new Cookie("color", "rojo");
    response.addCookie(galletaColor);

}
```

En este ejemplo podemos ver una cookie simple con todas sus opciones por defecto.



< > Manejando las cookies

Además, una cookie puede tener los siguientes parámetros:

- **Dominio** : Establece el nombre de dominio (o dominios) para el cual una cookie es válida. Se establece a través del método `setDomain(String dominio)` de la clase `Cookie`.
- **Path** : Establece la uri especifica en la que la cookie será válida (e.g: /admin). Es útil si no queremos exponer la cookie en toda la aplicación. Se establece con el método `setPath(String path)` de la clase `Cookie`.
- **Expiración** : Indica cuánto debe durar una cookie en el navegador. Este valor se establece en segundos a través del método `setMaxAge(int segundos)` de la clase `Cookie`.



< > Manejando las cookies

- **HostOnly** : Esta propiedad es muy importante en términos de seguridad, ya que establece si una cookie podrá ser leída por el cliente o solo por el servidor. Por eso **HostOnly** debería ser **true** siempre que el dominio desde el cual se está intentando leer no sea igual al dominio de origen establecido.
- **Secure** : Otra parámetro muy importante que indica que la cookie en cuestión puede ser leída únicamente a través de un protocolo seguro HTTPS o SSL. Y se establece con el método **setSecure(boolean isSecure)** de la clase Cookie.
- **HttpOnly** : Es similar a HostOnly excepto que esta si se puede manipular. Funciona como una medida de seguridad extra a la especificación establecida. Java al ser un lenguaje de clase mundial ofrece el método **setHttpOnly(boolean isHttpOnly)** para estos efectos.





Ejercicio N° 1

Servlet

< > Servlet

Manos a la obra: 🙌

Debes crear la estructura de un Servlet para poder registrar un usuario. En este caso, es necesario únicamente crear la clase 'UserServlet', y la declaración (no implementación) de los métodos doGet y doPost.

Consigna: ✍️

- 1- En el proyecto AlkeWallet, crear el servlet UserServlet
- 2- Agregar la anotación necesaria, mapeando la URL ("/userServlet")
- 3- Crear los métodos doGet y doPost.

Tiempo 🕒: 20 minutos



➤ Compartiendo información entre Servlets

< > Compartiendo información

En general, **compartir información entre servlets puede ser una forma eficaz de mejorar la funcionalidad, el rendimiento y la seguridad de su aplicación.**

Algunas razones por las que es útil compartir información entre servlets:

- **Para simplificar el código** : se puede evitar tener que pasar la información repetidamente entre los servlets. Esto puede simplificar el código y hacerlo más fácil de mantener.
- **Para mejorar el rendimiento** : se puede evitar tener que volver a cargar la información de la base de datos o de otros recursos. Esto puede mejorar el rendimiento de la aplicación.
- **Para aumentar la seguridad** : se puede restringir el acceso a la información a los usuarios autorizados. Esto puede ayudar a proteger su información contra el acceso no autorizado.



< > Compartiendo información

Algunas **formas de compartir información entre servlets** J2EE:

- **Atributos del contexto** : Los atributos del contexto de aplicación son compartidos por todos los servlets y JSP que se ejecutan en la misma aplicación en el servidor.

Para agregar un atributo al contexto, se utiliza el método `setAttribute()` en el objeto `ServletContext`. Para obtener un atributo del contexto, se utiliza el método `getAttribute()`.

```
// En el primer servlet
String dato = "Hola desde el primer servlet";
ServletContext context = getServletContext();
context.setAttribute("datoCompartido", dato);
```

```
// En el segundo servlet (segundoServlet)
ServletContext context = getServletContext();
String dato = (String) context.getAttribute("datoCompartido");
```



< > Compartiendo información

- **Sesiones** : los atributos de sesión son específicos para cada usuario y permanecen disponibles durante toda la sesión, incluso si el usuario navega entre diferentes páginas o servlets.

Para crear una sesión, se utiliza el método `getSession()` en el objeto `HttpServletRequest`. Para agregar información a una sesión, se utiliza el método `setAttribute()` en el objeto `HttpSession`. Para obtener información de una sesión, se utiliza el método `getAttribute()`.

```
// En el primer servlet
String dato = "Hola desde el primer servlet";
HttpSession session = request.getSession();
session.setAttribute("datoCompartido", dato);
```

```
// En el segundo servlet (segundoServlet)
HttpSession session = request.getSession();
String dato = (String) session.getAttribute("datoCompartido");
```



< > Compartiendo información

- **Atributos de solicitud:** Cuando un servlet envía una solicitud a otro servlet o JSP, puede establecer atributos en la solicitud que serán accesibles por el servlet de destino.

```
// En el primer servlet
String dato = "Hola desde el primer servlet";
request.setAttribute("datoCompartido", dato);
RequestDispatcher dispatcher = request.getRequestDispatcher("/segundoServlet");
dispatcher.forward(request, response);
```

```
// En el segundo servlet (segundoServlet)
String dato = (String) request.getAttribute("datoCompartido");
```



LIVE CODING

Ejemplo en vivo

Creando Servlets:

Vamos a crear un Servlet básico con los métodos vacíos para comenzar a compartir información entre servlets :

- 1- Crear un servlet `CurrencyServlet` que tenga declarados métodos para procesar solicitudes GET y POST.
- 2- Comunicar el nuevo servlet con `UserServlet` y compartir el nombre de usuario a través de atributos del contexto, sesiones, atributos de solicitud.

Tiempo: 25 minutos

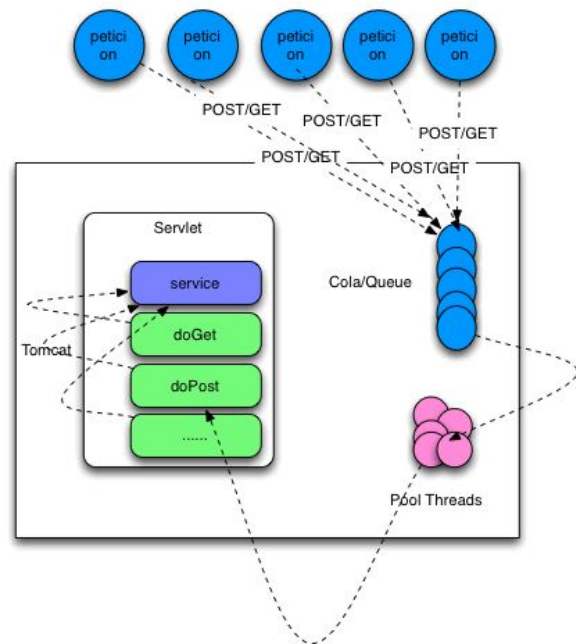
➤ Concurrencia entre Servlets

< > Concurrencia entre Servlets

¿Qué es?

La concurrencia se refiere a la ejecución simultánea de múltiples hilos o procesos que intentan acceder y modificar recursos compartidos.

Es un tema importante a tener en cuenta cuando trabajas con aplicaciones web, especialmente cuando varios hilos pueden interactuar con los mismos servlets al mismo tiempo.



< > Concurrencia entre Servlets

Se puede presentar en varias situaciones:

1. **Solicitudes concurrentes** : Cuando varios clientes (navegadores u otras aplicaciones) envían solicitudes HTTP al mismo servlet al mismo tiempo. Cada solicitud se maneja en un hilo separado en el servidor, lo que significa que múltiples hilos pueden ejecutar el mismo servlet simultáneamente.
2. **Recursos compartidos** : Si varios servlets comparten recursos globales, como variables estáticas o atributos de contexto de aplicación, podría haber problemas de concurrencia cuando varios hilos intentan acceder y modificar esos recursos compartidos al mismo tiempo.
3. **Sesiones** : Cuando varios usuarios interactúan con la misma aplicación web y utilizan sesiones para mantener información específica del usuario, es posible que varios hilos intenten acceder y modificar la misma sesión al mismo tiempo.



< > Concurrency entre Servlets

Algunas estrategias para manejar la concurrencia en servlets incluyen:

- **Sincronización** : Utiliza bloqueos (`synchronized`) para garantizar que solo un hilo pueda acceder a recursos compartidos en un momento dado.
- **Evitar recursos compartidos** : Diseña tu aplicación para evitar el uso excesivo de recursos compartidos (como variables globales).
- **Uso de sesiones seguras**: Asegúrate de que las sesiones sean seguras para ser accedidas y modificadas por múltiples hilos al mismo tiempo. Utiliza las sesiones correctamente y evita la posibilidad de pérdida de datos o inconsistencias.
- **Pruebas y análisis** : Realiza prueba para identificar posibles problemas de concurrencia. Utiliza herramientas de análisis de concurrencia y asegúrate de que tu aplicación funcione de manera correcta y predecible en entornos concurrentes.





Ejercicio N° 1

Dialogando

< > Dialogando

Manos a la obra: 🙌

En este caso, es necesario crear la clase 'CurrencyServlet', y la declaración (no implementación) de los métodos doGet y doPost.

Consigna: 📝

- 1- En el proyecto AlkeWallet, crear un servlet llamado CurrencyServlet.
- 2- Compartir la información del nombre de usuario de UserServlet con el CurrencyServlet a través del uso de Sesiones y de atributos de solicitud.

Tiempo 🕒: 30 minutos



➤ Controlando parámetros de un GET request

< > **Controlando parámetros de un GET request**

Si llamamos un servlet desde un formulario HTML, podremos hacerlo de dos formas: GET y POST. Con la primera los parámetros del formulario están incluidos la url que se utiliza para invocar el servlet y en el segundo caso los parámetros se almacenan en un buffer especial del servidor.

Para procesar el primer tipo de peticiones (GET) está el método doGet. La implementación por defecto del método service es capaz de determinar el tipo de petición HTTP que en un momento dado recibe el servlet. Una vez identificada, llama o al método doGet o al doPost según el caso.



< > **Controlando parámetros de un GET request**

Para controlar los parámetros en un GET request:

- Primero debemos obtener el valor de los parámetros en el servlet en el método doGet() utilizando el objeto HttpServletRequest.
- Podemos acceder a los parámetros mediante los métodos getParameter() o getParameterValues().
- Luego solo resta procesar y validar los parámetros según sea necesario para cada aplicación.



< > Controlando parámetros de un GET request

Veamos un ejemplo de cómo controlar los parámetros de una solicitud GET en un servlet:

Supongamos que tienes un servlet llamado MainServlet que maneja una solicitud GET con dos parámetros, "nombre" y "edad". El servlet mostrará estos parámetros en la respuesta al cliente.

En este caso, es como escribir código HTML en nuestro archivo Java.

```
@WebServlet("/mainServlet")
public class MainServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Obtenemos el valor del parámetro "nombre" de la solicitud GET
        String nombre = request.getParameter("nombre");

        // Obtenemos el valor del parámetro "edad" de la solicitud GET
        String edad = request.getParameter("edad");

        // Procesamos y validamos los parámetros según sea necesario
        // Por ejemplo, podríamos verificar si los parámetros son nulos o vacíos

        // Creamos la respuesta para el cliente
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().println("<html><body>");
        response.getWriter().println("<h1>Información recibida:</h1>");
        response.getWriter().println("<p>Nombre: " + nombre + "</p>");
        response.getWriter().println("<p>Edad: " + edad + "</p>");
        response.getWriter().println("</body></html>");

    }
}
```



< > Controlando parámetros de un GET request

Este método recibe los parámetros dados por el cliente a través de la clase `HttpServletRequest` y encapsula la respuesta que se le dará al cliente a través de la clase `HttpServletResponse`.

El servlet puede retornar al cliente cualquier tipo de información, desde texto plano hasta un ejecutable, por lo que es necesario señalar inicialmente qué tipo de respuesta se dará a través del método `setContentType`.

```
@WebServlet("/mainServlet")
public class MainServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Obtenemos el valor del parámetro "nombre" de la solicitud GET
        String nombre = request.getParameter("nombre");

        // Obtenemos el valor del parámetro "edad" de la solicitud GET
        String edad = request.getParameter("edad");

        // Procesamos y validamos los parámetros según sea necesario
        // Por ejemplo, podríamos verificar si los parámetros son nulos o vacíos

        // Creamos la respuesta para el cliente
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().println("<html><body>");
        response.getWriter().println("<h1>Información recibida:</h1>");
        response.getWriter().println("<p>Nombre: " + nombre + "</p>");
        response.getWriter().println("<p>Edad: " + edad + "</p>");
        response.getWriter().println("</body></html>");

    }
}
```



< > Controlando parámetros de un GET request

Entonces, el objeto **request** tiene información enviada en la petición.

El objeto **response** permite crear la respuesta

- **setContentType()**: permite indicar el tipo MIME de lo que se va a generar (por ejemplo, "text/html")
- **getWriter()**: devuelve el **PrintWriter** donde escribir lo que se genere

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/Saludo")
public class Saludo extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
     */
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        PrintWriter out = response.getWriter();
        out.println("¡Bienvenido al mundo de los servlets!");
    }
}
```

URL relativo a la webapp

Método para tratar la recepción de un HTTP GET

Salida generada para enviar al navegador

Generación de texto

< > Controlando parámetros de un GET request

El método `doGet()` envía los parámetros de forma explícita junto a la página, mostrando en la barra de navegación los parámetros y sus valores. Son esas largas cadenas que aparecen en algunas páginas en nuestra barra de navegación, luego del llamado al servlet, por ejemplo:

`/buscar?id=1806&valor=0987&texto=todo&...`

Las cadenas de la URL tienen la siguiente forma de referencia:

`parametro1=valor1¶metro2=valor2&...¶metroN=valorN.`

Es decir es una concatenación de pares **parámetro-valor** a través del operador '&'.



< > Controlando parámetros de un GET request

Las peticiones HTTP GET tienen un **límite en la cantidad de caracteres que pueden aceptar en el URL** . Si se envían los datos de un formulario muy extenso mediante HTTP GET pueden producirse errores por este motivo, por lo que habría que utilizar el método HTTP POST.

Se suele decir que el método GET es seguro e idempotente :

Seguro , porque no tiene ningún efecto secundario del cual pueda considerarse al usuario responsable del mismo. Es decir, por ejemplo, una llamada del método GET no debe ser capaz en teoría de alterar una base de datos. GET debería servir únicamente para obtener información.

Idempotente , porque puede ser llamado tantas veces como se quiera de una forma segura.

Es como si GET fuera algo así como **ver pero no tocar**.



LIVE CODING

Ejemplo en vivo

Primer método doGet():

Vamos a crear un Servlet básico con los métodos vacíos para comenzar a implementar un método doGet :

- 1- Crear un servlet CurrencyServlet con el método para procesar solicitudes GET que reciba un importe y el tipo de moneda a extraer.
- 2- Se deben mostrar los datos del usuario, el importe y el tipo de moneda seleccionado.

Tiempo: 30 minutos



Ejercicio N° 1

GET request

< > GET request

Manos a la obra: 🙌

Debes crear la estructura de un Servlet para poder registrar un usuario. En la clase 'UserServlet', implementar el método doGet.

Consigna: ✍️

- 1- En el proyecto AlkeWallet, trabajar sobre un servlet llamado UserServlet
- 2- Crear un formulario de login en doGet() y responder un saludo personalizado con el nombre de usuario que viaja en el formulario.

Tiempo 🕒: 30 minutos





Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprender la configuración e implementación de los Servlets**
- ✓ **Conocer el manejo de sesiones y de cookies**
- ✓ **Comprender cómo compartir información entre servlets**
- ✓ **Conocer el concepto de concurrencia**
- ✓ **Comprender cómo controlar los parámetros de un GET request**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

